Codename DataMedic

An Auditable AI Agent for Data Reliability

Formal Project Report

Nebius × NVIDIA Global AI Hackathon

Best Apps and Agents Track

Prepared by Team DataMedic

Project Lead: Jason Thai

23 October 2026

Table of Contents

Table of Contents 2
Abstract 4
1. Introduction 5
1.1 Background 5
1.2 Objectives 5
1.3 Scope 5
2. Problem Statement 5
3. Proposed Solution 6
3.1 Overview 6
3.2 Workflow 6
3.3 Issue Coverage 6
4. System Architecture 7
4.1 Design Principles 7
4.2 Components 8
4.3 Data Flow 9
5. The Neural Network 9
5.1 Model Selection 9
5.2 Model Architecture 9
5.3 Inference Process 10
5.4 Integration within DataMedic 10
5.5 Cost 11
6. Safety and Governance 11
7. Data Health Score 12
8. Implementation and Deployment 12
8.1 Technology Stack 12
8.2 User Interface 13
8.3 Deployment and Security 13
9. Evaluation 13
9.1 Method 13
9.2 Detection Accuracy 13
9.3 Repair Effectiveness 13
9.4 Robustness, Speed and Cost 14
10. Limitations and Future Work 14
11. Conclusion 14

12. Project Team 15
References 16
Appendix A. Example Model Input and Output 17

Abstract

This report presents Codename DataMedic, an artificial-intelligence agent developed for the Nebius × NVIDIA Global AI Hackathon (Best Apps and Agents track) to improve the reliability of tabular data. DataMedic detects fourteen categories of data-quality defect, obtains a proposed repair for each from the NVIDIA Nemotron 3 Nano large language model served through Nebius Token Factory, and applies only those repairs that satisfy deterministic safety rules. Every applied repair is verified and reversed if it fails, uncertain cases are referred to a human reviewer, and all changes are recorded in an audit trail. A five-dimension Data Health Score quantifies quality before and after processing. In a controlled evaluation on a synthetic dataset containing 112 planted errors, the system detected all 112 (100 per cent) and raised the Health Score from 92.9 to 97.0 through automatic repairs alone, at a model cost of approximately USD 0.001 per analysis. The results indicate that restricting a language model to a proposing role, while delegating execution and verification to deterministic code, yields data cleaning that is both efficient and auditable.

1. Introduction

1.1 Background

The Nebius × NVIDIA Global AI Hackathon invites teams to build functioning artificial-intelligence applications using Nebius Token Factory together with NVIDIA Nemotron or other NVIDIA open-source models. Submissions close on 30 October 2026 and must include a public video of no more than three minutes demonstrating the working project and its use of these technologies. DataMedic was entered in the Best Apps and Agents track.

1.2 Objectives

The project set out to build an agent that (a) identifies data-quality defects automatically, (b) uses a neural network to reason about appropriate repairs, (c) applies repairs only where the evidence is strong, (d) refers uncertain cases to a person rather than guessing, and (e) provides verifiable evidence of every change. The guiding principle was: automate when the evidence is strong, and ask when it is not.

1.3 Scope

This report describes the problem addressed, the proposed solution, the system architecture, the neural network and its integration, the safety model, the scoring methodology, the implementation and deployment, the evaluation results, and the limitations and planned future work.

2. Problem Statement

Organisations routinely base decisions on spreadsheets and data exports that contain defects. Typical examples include the same value recorded in several forms (for instance "ON", "Ont." and "ontario" for Ontario), numeric values stored as formatted text ("$1,200"), dates recorded in mixed formats, some of which are ambiguous (03/04/26 may denote 3 April or 4 March), duplicate records and identifiers, missing values, implausible values and violations of business rules.

Manual correction of such defects is slow, repetitive and itself prone to error. Fully automated cleaning tools are faster but frequently resolve uncertainty by silent assumption, for example by choosing a day–month order for an ambiguous date or by imputing a missing value, and they seldom record what was changed. The resulting data may appear clean while being neither trustworthy nor explainable. The problem addressed by DataMedic is therefore how to clean data rapidly without guessing, while providing evidence for every change.

3. Proposed Solution

3.1 Overview

DataMedic is an AI data-reliability agent that processes any CSV file through a controlled workflow. A neural network is used for reasoning, whereas all modifications to data are performed, verified and recorded by deterministic Python code. The original file is never modified.

3.2 Workflow

The workflow comprises eight stages:

1. Detection: every column is profiled and data-quality issues are identified.
2. Reasoning: a compact summary of each issue is submitted to NVIDIA Nemotron, which returns a proposed repair, a confidence score and a rationale.
3. Validation of proposals: code-level guardrails determine whether each proposal may be executed automatically.
4. Repair: approved, high-confidence repairs are executed by deterministic repair tools.
5. Verification: the data is profiled again, and any repair that has not resolved its issue is rolled back.
6. Human review: risky or uncertain issues are presented to a reviewer, who may approve the proposed repair, retain the data unchanged, decline the proposal or enter a corrected value.
7. Audit: every repair, rollback and decision is recorded with its previous and new values.
8. Reporting: a cleaned dataset, an audit report and a before-and-after Data Health Score are produced.

3.3 Issue Coverage

Table 1 lists the fourteen issue categories detected and the treatment applied to each. Five categories that involve formatting only may be repaired automatically; the remaining nine always require human judgement.

Table 1. Issue categories and their treatment

Issue category	Example	Treatment
Category variants	"AB" and "Alberta" in one column	Automatic repair
Extra whitespace	" Ontario "	Automatic repair
Numbers stored as text	"$1,200"	Automatic repair
Mixed date formats	2026-01-05 and Jan 15 2026	Automatic repair
Non-standard date format	27/11/2021	Automatic repair
Ambiguous date	03/04/26	Reviewer enters the date
Missing values	Blank or "N/A"	Reviewer decides
Duplicate rows	The same order entered twice	Reviewer approves removal
Duplicate identifiers	One order number on two rows	Reviewer decides
Invalid value	Text in a numeric column	Reviewer decides
Suspicious negative value	Revenue of −450	Reviewer decides
Outlier	99,999 among values near 500	Reviewer decides
Future date	An order dated in the following year	Reviewer decides
Business-rule violation	Quantity of 0 where the minimum is 1	Reviewer decides

Business rules may be supplied as a JSON file supporting seven rule types (unique, not null, minimum, maximum, allowed values, Canadian province and pattern) or inferred automatically from the data. Users may also provide a one-sentence description of the dataset, which is passed to the model as business context.

4. System Architecture

4.1 Design Principles

The architecture separates reasoning from execution. The neural network proposes repairs but cannot modify data; deterministic code decides which proposals may run, applies them, verifies their effect and records the outcome. The web interface never edits data directly: it invokes the analysis pipeline for each new dataset and the agent module for each human decision.

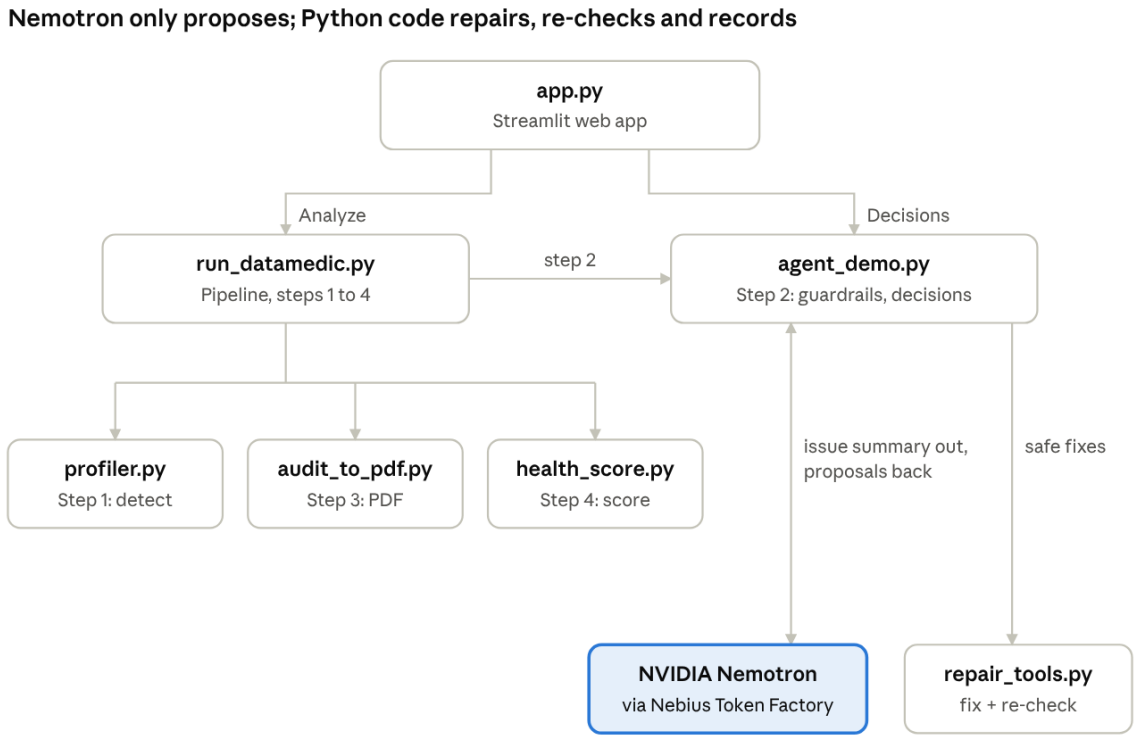


Figure 1. System architecture: principal components and the calls between them

4.2 Components

Table 2 summarises the responsibilities of each component.

Table 2. System components

Component	Responsibility
app.py	Streamlit web interface: upload, dataset context, rules, dashboard, decision cards, audit trail and downloads
run_datamedic.py	Orchestrates one analysis: validates the file, creates a results folder, selects rules and runs stages 1 to 4
profiler.py	Infers column types, detects issues, and evaluates and infers business rules
agent_demo.py	Constructs the prompt, calls Nemotron, applies guardrails and safe repairs, and processes human decisions
repair_tools.py	The only component that modifies data: six repair tools with apply, verify and rollback functions
health_score.py	Computes the five-dimension Data Health Score before and after processing
audit_to_pdf.py	Generates the audit report in PDF format
NVIDIA Nemotron 3 Nano	External neural network, accessed through Nebius Token Factory, that proposes one repair per issue

4.3 Data Flow

For each analysis, the user uploads a CSV file, optionally with a rules file and a dataset description. The pipeline confirms that the file is a readable table, creates a dedicated results folder and selects the applicable business rules. The profiler records every issue with a stable identifier, the affected rows, supporting evidence and example values. The agent submits the issues to Nemotron in batches of eight, evaluates each proposal against the guardrails, applies permitted repairs through the repair tools, re-profiles the data after each repair and rolls back any repair that fails. The audit report and Health Score are then generated, and the web interface displays results read directly from the run's output files. Each subsequent human decision invokes the agent with an approve, accept, reject or resolve command, after which the audit report and score are regenerated.

5. The Neural Network

5.1 Model Selection

DataMedic employs NVIDIA Nemotron 3 Nano 30B-A3B, an open large language model designed for agentic workloads such as reasoning and multi-step tool use. The model was selected for three reasons: its training for tool selection corresponds directly to DataMedic's task of choosing a repair tool and its parameters for each issue; its sparse mixture-of-experts design provides fast, low-cost inference relative to its total size; and it returns the structured JSON responses the system requires. The model is accessed through the OpenAI-compatible API of Nebius Token Factory and is used as published, without training or fine-tuning.

5.2 Model Architecture

Table 3 summarises the principal characteristics of the model as reported by NVIDIA.

Table 3. Characteristics of NVIDIA Nemotron 3 Nano 30B-A3B

Property	Value
Architecture	Hybrid of Mamba-2 and Transformer (grouped-query attention) layers with mixture-of-experts layers; 52 layers
Parameters	31.6 billion in total; approximately 3.2 billion active per token
Mixture of experts	128 routable experts and 2 shared experts; 6 routable experts selected per token
Context window	Up to 1 million tokens
Training	25 trillion tokens of pre-training, followed by post-training for reasoning, tool use, code, mathematics and instruction following
Availability	Open model weights published by NVIDIA

5.3 Inference Process

Figure 2 illustrates how the network processes a single DataMedic request. The prompt is first divided into tokens. Each token's representation then passes through a stack of 52 interleaved layers of three types. Mamba-2 layers maintain a running summary of the sequence whose cost grows only linearly with input length. Attention layers allow each token to refer to any earlier token, which enables the model to relate an issue to the corresponding tool description and business rule. In each mixture-of-experts layer, a router assigns every token to 6 of 128 expert networks in addition to 2 shared experts, so that only about 3.2 billion of the 31.6 billion parameters are active for any token. The response is generated one token at a time, each new token being fed back as input. A sampling temperature of 0.1 is used so that outputs are close to deterministic. Any reasoning text is removed, and the resulting JSON is passed to the guardrails.

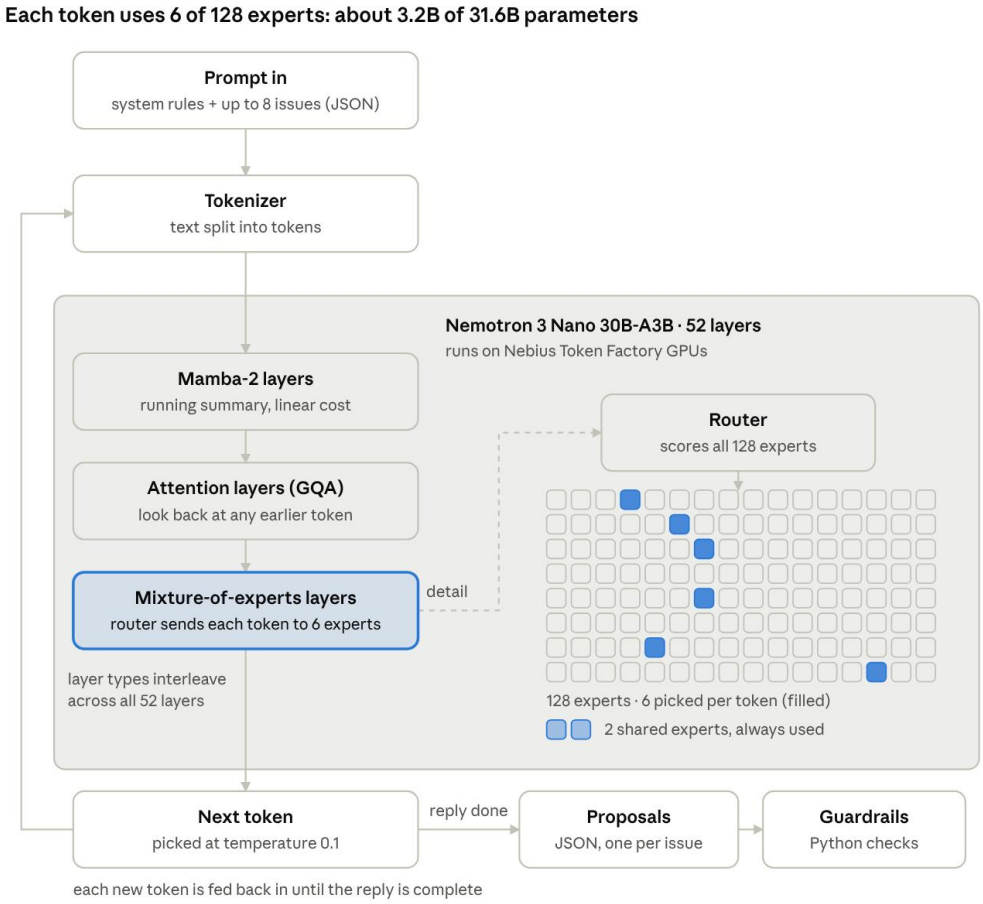


Figure 2. Inference within Nemotron 3 Nano during one DataMedic request

5.4 Integration within DataMedic

The neural network is invoked at a single point in the workflow (stage 2), once for each batch of up to eight issues. Profiling, guardrails, repair, verification, scoring, reporting and the processing of human

decisions are all performed by deterministic Python code. Table 4 describes the messages exchanged with the model.

Table 4. Messages exchanged with the model

Message	Contents
System prompt (input)	The model's role, the six permitted repair tools and their parameters, and constraints: never invent data, never guess ambiguous dates, reply in JSON only
User message (input)	Dataset context, column types, business rules and up to eight issue summaries, each with a small number of example values
Response (output)	One proposal per issue: repair tool and parameters, confidence between 0 and 1, a one-line rationale and whether approval is required

The complete dataset is never transmitted; issue summaries contain only a small number of example cell values. An example request and response are provided in Appendix A.

5.5 Cost

At Token Factory prices of USD 0.06 per million input tokens and USD 0.24 per million output tokens, a typical analysis consumes a few thousand tokens and costs approximately USD 0.001.

6. Safety and Governance

DataMedic does not permit the model to modify data. Every proposal is subject to the controls in Table 5, which are enforced in code irrespective of the model's recommendation.

Table 5. Safety controls

Control	Effect
Mandatory review of risky categories	Missing values, duplicates, duplicate identifiers, outliers, negative values, ambiguous dates, future dates and rule violations are never repaired automatically
Confidence threshold	Automatic repairs require a confidence of at least 0.90
Tool suitability	A tool that cannot resolve the issue category is replaced by a request for review
Valid values only	Category mappings may use only values that occur in the data
Approval of destructive actions	Removal of rows always requires a reviewer
Verification and rollback	Data is re-profiled after each repair, and the repair is reversed if the issue persists
Validation of human input	Values entered by reviewers are checked and rejected if they introduce a new defect
Preservation of source data	The uploaded file is never altered; each run writes to its own folder

Reviewers may approve a proposed repair, retain the data unchanged, decline a proposal with a reason, or enter a corrected value for a specific row. Each audit entry is written by the code that performed the action and records the issue, the proposed repair, its confidence, the tool used, the source of approval, the affected row, the previous and new values, the verification result and any rollback.

7. Data Health Score

The Data Health Score expresses data quality on a scale of 0 to 100 as the weighted proportion of rows that pass five checks, as defined in Table 6.

Table 6. Dimensions of the Data Health Score

Dimension	Weight	A row fails if it contains
Completeness	25%	A missing value
Uniqueness	20%	A full duplicate of another row, or a repeated identifier
Validity	20%	A number or date that is not in standard format
Consistency	15%	A category value written in a non-standard form
Business rules	20%	A violation of a business rule

Dimensions with nothing to measure are excluded and the remaining weights are rescaled proportionally. Scores are graded as Healthy (95 or above), Fair (80 to 94.9), Poor (60 to 79.9) or Critical (below 60). Scores before and after processing are computed using the same column types to ensure comparability, and any dimension in which the number of failing rows increases is reported as a regression.

8. Implementation and Deployment

8.1 Technology Stack

Table 7 lists the technologies used.

Table 7. Technology stack

Layer	Technology
Reasoning	NVIDIA Nemotron 3 Nano 30B-A3B
Model hosting	Nebius Token Factory (OpenAI-compatible API)
Programming language	Python 3
Data processing	pandas
Web application	Streamlit
Report generation	ReportLab
Hosting	Streamlit Community Cloud (private application)
Source control	GitHub

The codebase comprises approximately 3,150 lines of Python in nine modules, together with example business-rule files for retail, employee and passenger datasets.

8.2 User Interface

The web application provides a start page for uploading data, describing the dataset and supplying optional rules; a dashboard presenting the Health Score, the number of issues found, repaired and awaiting review, before-and-after results for each dimension, and a log of agent activity; decision cards with a severity filter and the four review actions; and pages for the audit trail, the cleaned data and downloads of the cleaned CSV, audit report and audit log. Previous analyses can be reopened, and invalid files produce a plain-language message.

8.3 Deployment and Security

The application is hosted as a private Streamlit application accessible only to invited users. The Nebius API key is stored as a hosted secret and is never committed to source control. The repository excludes credentials, results and uploaded files, and detailed error information is hidden from users. The same pipeline can also be executed from the command line.

9. Evaluation

9.1 Method

Detection accuracy was assessed on a synthetic retail dataset of 305 rows into which 112 errors of thirteen kinds were planted at known locations, with an accompanying answer key. Repair effectiveness was assessed by comparing Health Scores before and after automatic repairs on four datasets. Robustness was assessed through a 23-case test plan.

9.2 Detection Accuracy

DataMedic detected all 112 planted errors (100 per cent), both with manually written business rules and with automatically inferred rules. This result applies to the controlled evaluation and does not guarantee detection of every possible defect in arbitrary real-world data.

9.3 Repair Effectiveness

Table 8 reports Health Scores before processing and after automatic repairs, prior to any human decisions.

Table 8. Health Scores before and after automatic repairs

Dataset	Rows	Issues	Before	After automatic repairs
Synthetic retail (planted errors)	305	23	92.9 (Fair)	97.0 (Healthy)
Retail sample	20	16	79.2 (Poor)	89.5 (Fair)
Employee records	1,020	5	54.3 (Critical)	67.1 (Poor)
Titanic passengers	891	6	80.1 (Fair)	80.1 (Fair)

For the employee dataset, 654 dates were converted to a standard format automatically. Following human review of the synthetic dataset the score rose to approximately 98, and all six ambiguous dates resolved by the reviewer agreed with the answer key. The Titanic score was unchanged by design, because its defects consist of genuinely missing ages and cabin numbers, which DataMedic does not invent.

9.4 Robustness, Speed and Cost

The test plan covered the complete workflow, user-supplied and mismatched rules files, a 1,020-row file, empty, header-only and non-CSV files, accented text, small screens and concurrent users. Invalid files produced clear messages rather than errors. A typical analysis completed in 20 to 40 seconds at a model cost of approximately USD 0.001.

10. Limitations and Future Work

The current version has several limitations, summarised in Table 9 together with the planned improvements. None of these limitations results in incorrect changes to data; they affect how frequently a reviewer is consulted or how the score should be interpreted. In addition, the results reported in Section 9 were obtained with the team's latest code, and the public repository should be updated with the latest versions of profiler.py, run_datamedic.py and app.py so that it reproduces these results.

Table 9. Limitations and planned improvements

Limitation	Planned improvement
Outlier detection flags some genuine large values (17 of 20 flagged values in the synthetic dataset were legitimate)	Detect outliers within groups, such as per product
Completeness counts every missing cell, so optional columns reduce the score	Allow optional columns to be designated or weighted
The row-based score may appear lenient for very defective files	Report an error-density measure alongside the score
Future dates are flagged but not scored	Include them in the business-rules dimension
The web application accepts CSV files only	Support Excel files and direct database connections
Results on the hosted application are shared by all users and cleared on restart	Provide per-user workspaces with saved history
Rules and decisions are not reused between analyses	Store team rules and previous decisions for reuse

11. Conclusion

DataMedic demonstrates that an AI agent can make data cleaning both faster and more trustworthy when the language model is confined to a reasoning role. NVIDIA Nemotron 3 Nano, served efficiently through Nebius Token Factory, proposes repairs at a cost of about one tenth of a cent per analysis, while deterministic code enforces safety, verifies every change and maintains a complete audit trail. Uncertain

cases are referred to a person rather than resolved by assumption. In evaluation, the system detected every planted error and measurably improved data health, offering a practical and auditable approach to AI-assisted data engineering.

12. Project Team

Table 10. Project team

Member	Role
Jason Thai	Project Lead; Reasoning Lead; Profiling, Validation and Health Score; Frontend; Product and Pitch
Ria Chahar	Agent Architect; Repair Tools; Backend and Infrastructure; Product and Pitch
Omer Irfan	Application Testing and Quality Assurance
Prabal Pratap Sharma, Nashwa Meher Ahmed Lnu	Registered team members (not involved in development)

References

1. NVIDIA (2025). Nemotron 3 Nano: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. arXiv:2512.20848. https://arxiv.org/html/2512.20848v1
2. NVIDIA. NVIDIA Nemotron 3 Nano Technical Report. https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Nano-Technical-Report.pdf
3. NVIDIA. Nemotron 3 Nano 30B-A3B Model Overview. https://docs.api.nvidia.com/nim/reference/nvidia-nemotron-3-nano-30b-a3b
4. Codename DataMedic source code. https://github.com/takuto02/Nebius-x-NVIDIA-Global-AI-Hackathon-Codename-DataMedic

Appendix A. Example Model Input and Output

The following issue summary, taken from the retail sample dataset, illustrates the input submitted to the model for one issue.

```
{
 "id": "iss_007",
 "key": "Province:category_variants:Alberta",
 "column": "Province",
 "type": "category_variants",
 "severity": "medium",
 "row_count": 1,
 "evidence": "2 spellings of the same value: {'Alberta': 2, 'AB': 1} -
> likely 'Alberta' (Canadian province)",
 "values": {"Alberta": 2, "AB": 1},
 "rule_id": null,
 "suggested_standard": "Alberta"
}
```

An illustrative response in the required format is shown below. Because the issue key is valid, the value "AB" occurs in the data, the tool is suitable for a category-variant issue and the confidence exceeds 0.90, the guardrails permit this repair to be applied automatically and then verified.

```
{"proposals": [{
  "key": "Province:category_variants:Alberta",
  "issue_id": "iss_007",
  "rule_id": null,
  "tool": "standardize_category",
  "params": {"column": "Province", "mapping": {"AB": "Alberta"}},
  "confidence": 0.95,
  "rationale": "AB is the standard abbreviation for Alberta.",
  "needs_approval": false
}]}
```